```
# Command to generate migration:
# npx sequelize-cli migration:generate --name create-users-table
```

To use sequelize with PostgreSQL:

```
npm install express sequelize pg pg-hstore
npm install --save-dev sequelize-cli nodemon
```

To initialize sequelize project:

```
# Initialize Sequelize
npx sequelize-cli init
```

To generate model using sequelize:

```
# Generate User model with migration
npx sequelize-cli model:generate --name User --attributes
username:string,email:string

# Generate Profile model with migration
npx sequelize-cli model:generate --name Profile --attributes
userId:integer,firstName:string,lastName:string,bio:text,dateOfBirth:date
```

To run all migrations:

```
# Run all pending migrations
npx sequelize-cli db:migrate
```

```
# Run specific migration
npx sequelize-cli db:migrate --name 20231001000001-create-users-table.js
```

```
# Undo last migration
npx sequelize-cli db:migrate:undo
```

```
# Undo all migrations
npx sequelize-cli db:migrate:undo:all
            ------------------------------------------------------------

# Check migration status
npx sequelize-cli db:migrate:status
            ------------------------------------------------------------

# Generate seeder
npx sequelize-cli seed:generate --name demo-data
            ------------------------------------------------------------

# Run all seeders
npx sequelize-cli db:seed:all
            ------------------------------------------------------------

# Run specific seeder
npx sequelize-cli db:seed --seed 20231001000001-demo-data.js
            ------------------------------------------------------------

# Undo all seeders
npx sequelize-cli db:seed:undo:all
            ------------------------------------------------------------

# Undo specific seeder
npx sequelize-cli db:seed:undo --seed 20231001000001-demo-data.js
            ------------------------------------------------------------

# Create database
npx sequelize-cli db:create
            ------------------------------------------------------------

# Drop database
npx sequelize-cli db:drop
            ------------------------------------------------------------

# Reset database (drop, create, migrate)
npx sequelize-cli db:migrate:reset
            ------------------------------------------------------------
```

The post model:

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class Post extends Model {
    /**
     * Define associations here.
     * Many-to-One: Post belongs to User
     * Many-to-Many: Post has many Tags through PostTags
     */
    static associate(models) {
      // Post belongs to a User
      Post.belongsTo(models.User, {
        foreignKey: 'userId',
        as: 'author', // Alias for eager loading
        onDelete: 'CASCADE',
        onUpdate: 'CASCADE'
      });

      // Many-to-Many relationship with Tags
      Post.belongsToMany(models.Tag, {
        through: models.PostTag, // Junction table
        foreignKey: 'postId',
        otherKey: 'tagId',
        as: 'tags', // Alias for eager loading
        onDelete: 'CASCADE',
        onUpdate: 'CASCADE'
      });
    }
  }

  Post.init({
    id: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true
    },
    title: {
      type: DataTypes.STRING,
      allowNull: false,
      validate: {
        len: {
          args: [5, 200],
          msg: 'Title must be between 5 and 200 characters'
```

```
        }
      }
    },
    content: {
      type: DataTypes.TEXT,
      allowNull: false,
      validate: {
        len: {
          args: [10, 5000],
          msg: 'Content must be between 10 and 5000 characters'
        }
      }
    },
    userId: {
      type: DataTypes.INTEGER,
      allowNull: false,
      references: {
        model: 'Users',
        key: 'id'
      }
    },
    isPublished: {
      type: DataTypes.BOOLEAN,
      defaultValue: false
    }
  }, {
    sequelize,
    modelName: 'Post',
    tableName: 'Posts',
    timestamps: true
  });

  return Post;
};
```

The tag model is:

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class Tag extends Model {
    /**
     * Define associations here.
     * Many-to-Many: Tag has many Posts through PostTags
     */
    static associate(models) {
      // Many-to-Many relationship with Posts
      Tag.belongsToMany(models.Post, {
        through: models.PostTag, // Junction table
        foreignKey: 'tagId',
        otherKey: 'postId',
        as: 'posts', // Alias for eager loading
        onDelete: 'CASCADE',
        onUpdate: 'CASCADE'
      });
    }
  }

  Tag.init({
    id: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true
    },
    name: {
      type: DataTypes.STRING,
      allowNull: false,
      unique: {
        msg: 'Tag name already exists'
      },
      validate: {
        len: {
          args: [2, 50],
          msg: 'Tag name must be between 2 and 50 characters'
        }
      }
    },
    description: {
      type: DataTypes.TEXT,
      validate: {
```

```
        len: {
          args: [0, 500],
          msg: 'Description cannot exceed 500 characters'
        }
      }
    }
  }, {
    sequelize,
    modelName: 'Tag',
    tableName: 'Tags',
    timestamps: true
  });

  return Tag;
};
```

The PostTag Model:

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class PostTag extends Model {
    /**
     * Define associations here.
     * This is a junction table for Many-to-Many relationship
     */
    static associate(models) {
      // PostTag belongs to Post
      PostTag.belongsTo(models.Post, {
        foreignKey: 'postId',
        as: 'post',
        onDelete: 'CASCADE',
        onUpdate: 'CASCADE'
      });

      // PostTag belongs to Tag
      PostTag.belongsTo(models.Tag, {
        foreignKey: 'tagId',
        as: 'tag',
        onDelete: 'CASCADE',
        onUpdate: 'CASCADE'
      });
    }
```

```
  }

  PostTag.init({
    id: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true
    },
    postId: {
      type: DataTypes.INTEGER,
      allowNull: false,
      references: {
        model: 'Posts',
        key: 'id'
      }
    },
    tagId: {
      type: DataTypes.INTEGER,
      allowNull: false,
      references: {
        model: 'Tags',
        key: 'id'
      }
    }
  }, {
    sequelize,
    modelName: 'PostTag',
    tableName: 'PostTags',
    timestamps: true,
    indexes: [
      {
        unique: true,
        fields: ['postId', 'tagId'], // Composite unique index
        name: 'unique_post_tag'
      }
    ]
  });

  return PostTag;
};
```

The user model is:

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class User extends Model {
    /**
     * Define associations here.
     * One-to-One: User has one Profile
     * One-to-Many: User has many Posts
     */
    static associate(models) {
      // One-to-One relationship with Profile
      User.hasOne(models.Profile, {
        foreignKey: 'userId',
        as: 'profile', // Alias for eager loading
        onDelete: 'CASCADE', // Delete profile when user is deleted
        onUpdate: 'CASCADE'
      });

      // One-to-Many relationship with Posts
      User.hasMany(models.Post, {
        foreignKey: 'userId',
        as: 'posts', // Alias for eager loading
        onDelete: 'CASCADE', // Delete all posts when user is deleted
        onUpdate: 'CASCADE'
      });
    }
  }

  User.init({
    id: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true
    },
    username: {
      type: DataTypes.STRING,
      allowNull: false,
      unique: {
        msg: 'Username already exists'
      },
      validate: {
        len: {
          args: [3, 30],
```

```
          msg: 'Username must be between 3 and 30 characters'
        }
      }
    },
    email: {
      type: DataTypes.STRING,
      allowNull: false,
      unique: {
        msg: 'Email already exists'
      },
      validate: {
        isEmail: {
          msg: 'Please provide a valid email address'
        }
      }
    }
  }, {
    sequelize,
    modelName: 'User',
    tableName: 'Users',
    timestamps: true,
    paranoid: false // Set to true if you want soft deletes
  });

  return User;
};
```

The Profile model:

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class Profile extends Model {
    /**
     * Define associations here.
     * One-to-One: Profile belongs to User
     */
    static associate(models) {
      // Profile belongs to a single User
      Profile.belongsTo(models.User, {
        foreignKey: 'userId',
        as: 'user', // Alias for eager loading
        onDelete: 'CASCADE',
```

```
      onUpdate: 'CASCADE'
    });
  }
}

Profile.init({
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  userId: {
    type: DataTypes.INTEGER,
    allowNull: false,
    unique: true, // Ensures one-to-one relationship
    references: {
      model: 'Users',
      key: 'id'
    }
  },
  firstName: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      len: {
        args: [2, 50],
        msg: 'First name must be between 2 and 50 characters'
      }
    }
  },
  lastName: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      len: {
        args: [2, 50],
        msg: 'Last name must be between 2 and 50 characters'
      }
    }
  },
  bio: {
    type: DataTypes.TEXT,
    validate: {
      len: {
        args: [0, 1000],
```

```
          msg: 'Bio cannot exceed 1000 characters'
        }
      }
    },
    dateOfBirth: {
      type: DataTypes.DATE,
      validate: {
        isDate: {
          msg: 'Please provide a valid date'
        }
      }
    }
  }, {
    sequelize,
    modelName: 'Profile',
    tableName: 'Profiles',
    timestamps: true
  });

  return Profile;
};
```